

ProbOS — Month 2, Week 8

Cross-Discipline Batch 2 + Month 2 Retrospective

Nisong Monyimba

July 4, 2026

Week 8 goal

Show that `ParticleFilter` (Week 5) generalises across domains the same way `MonteCarloEngine` was shown to in Month 1 Week 4: port the ED queue and clinical trial models to a sequential filtering setting. Close Month 2 with a retrospective and Month 3 plan.

Day-by-day plan

Day 1 – ED queue filtering setup

- Reuse `EDQueueModel` from `python/examples/week4_ed_queue.py` unmodified
- Design the inference problem: infer the TRUE (unknown) arrival rate λ from an observed queue-length time series, using `ParticleFilter`
- Construct a Gaussian observation log-likelihood on queue length, following the same pattern already established in `test_particle_filter.py` and the `/filter` endpoint

Day 2 – ED queue filtering validation

- Generate synthetic queue-length observations from a KNOWN true λ
- Confirm the particle filter’s posterior mean for λ converges toward the true value as more observations arrive
- `python/examples/week8_ed_queue_filter.py`

Day 3 – Clinical trial sequential monitoring setup

- Reuse `ClinicalTrialModel` from `python/examples/week4_clinical_trial.py` unmodified
- Replace Week 4’s batch nested-Monte-Carlo posterior-probability calculation with `ParticleFilter`, updating the posterior belief about treatment effect after EACH patient enrolls, rather than only at trial’s end

Day 4 – Clinical trial filtering validation

- Confirm the sequentially-updated posterior at the FINAL patient matches (or closely agrees with) Week 4’s batch calculation on the same trial data – a consistency check between the two inference approaches on the same model
- `python/examples/week8_clinical_trial_filter.py`

Day 5 – Test coverage

- `python/tests/test_week8_examples.py`: shape contracts, physical/statistical invariants, and validation against the respective ground truth for both new filtering examples – matching the standard already set for Week 4’s cross-discipline examples

Day 6 – `pre_week_audit.sh` + CI

- Run `pre_week_audit.sh` before considering the week complete
- Extend CI’s example-running steps to include the two new filtering scripts

Day 7 – Month 2 retrospective + Month 3 plan

- Write the Month 2 retrospective directly into `docs/monthly_plans/month2/main.tex`, alongside a Month 3 plan in `docs/monthly_plans/month3/main.tex`
- Retrospective for THIS week appended to this same document once the week’s work is actually complete, per the established plan-then-retrospective process

Exit criteria

Both filtering examples pass validation against their respective ground truth (known true λ for the ED queue; batch-calculation consistency for the clinical trial), with test coverage matching Month 1’s standard – shape contracts, physical invariants, and validation against a closed-form or independently-computed benchmark, not just “runs without crashing”.

What was actually accomplished (retrospective)

- `ParticleFilter.params`: a new public property exposing each particle’s fixed parameter values – genuine reusable kernel functionality (not a one-off workaround), needed to recover the posterior over PARAMETERS rather than only the posterior over state that `PFRresult.means/stds` already provided.
- `week8_ed_queue_filter.py`: infers the true (unknown) patient arrival rate λ from a noisy, observed queue-length time series. Starting from a deliberately WIDE Uniform(5, 15) prior, the filter’s posterior mean landed within 0.1–0.2 standard deviations of the true $\lambda = 10.0$ across multiple runs.
- `week8_clinical_trial_filter.py`: replaces Week 4’s batch nested-Monte-Carlo posterior calculation with sequential, patient-by-patient updating. The filter’s final posterior ($P(\text{treatment better}) = 0.6786$) agreed closely with Week 4’s exact Beta-Binomial batch calculation (0.6582) on the identical trial data.

- Both examples required introducing a purpose-built, domain-appropriate state-transition design rather than mechanically reusing Week 4’s forward-simulation models unchanged – the ED queue case needed a documented, narrowly-scoped exception to project doctrine around `np.random.seed()` (justified and defensively scoped with state save/restore); the clinical trial case needed a new `ClinicalTrialFilterModel` reflecting that real trial data is fully observed (deterministic state transitions), unlike the ED queue’s genuinely noisy observations.
- Two real bugs were found and fixed during development, not glossed over: a genuine reproducibility bug in the ED queue ground-truth generator (not actually seeding the global state `EDQueueModel` depends on internally), and an apples-to-oranges prior mismatch in the clinical trial validation (comparing results computed under two different priors, which produced a misleading 0.18 “disagreement” that was actually two different, individually-correct answers to two different problems).
- A real CI gap was found and closed: neither new example script was actually being run in CI despite this plan’s own Day 6 commitment – both are now genuinely executed in the `python-tests` and `integration` jobs.

Numbers at Week 8 close

- Python tests: 368/368 passing (350 + 18 new)
- C++ tests: 13/13 passing
- mypy strict: 0 errors (full `python/` tree)
- ruff: 0 warnings (full declared config)
- `pre_week_audit.sh`: 16/16 checks passed

Carried into Month 3

`ParticleFilter` is now validated across three genuinely different domains (an exact Kalman filter comparison in Week 5, a known-arrival-rate recovery in hospital operations, and an exact Beta-Binomial consistency check in clinical trials) – a stronger domain-agnosticism claim than Month 1’s forward-simulation-only demonstration. See `docs/monthly_plans/month2/main.tex` for the full Month 2 retrospective and `docs/monthly_plans/month3/main.tex` for what comes next.